

DWH Schema Design (Part 2)

Learning Objectives

- Declare fact grain, keys, measures, and aggregation rules
- Choose star, snowflake, or galaxy schemas by workload
- Model history, table behavior, and conformed dimensions
- Design partitioning and governance controls for production use

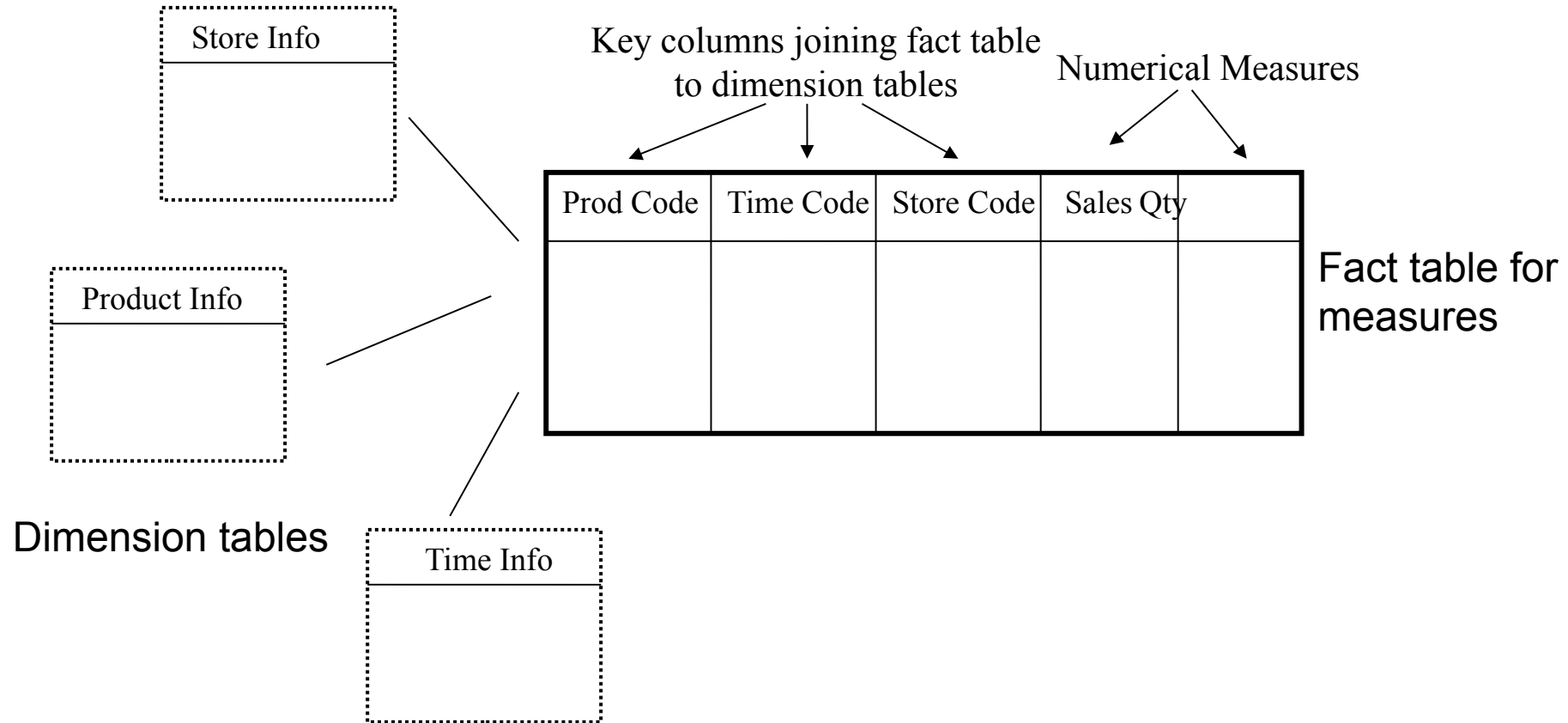
Why Schema Design Matters

- Schema design is where business meaning becomes executable
- Wrong grain or keys silently corrupt KPIs and downstream models
- Physical choices such as partitioning determine cost and latency
- Professional DWH design balances correctness, performance, and operability

Fact and Dimension Contracts

- Fact table: measurable business events at one declared grain
- Dimension table: descriptive context used to filter, group, and explain facts
- Measure: numeric value with explicit aggregation semantics
- Hierarchy: governed drill path such as day -> month -> quarter -> year

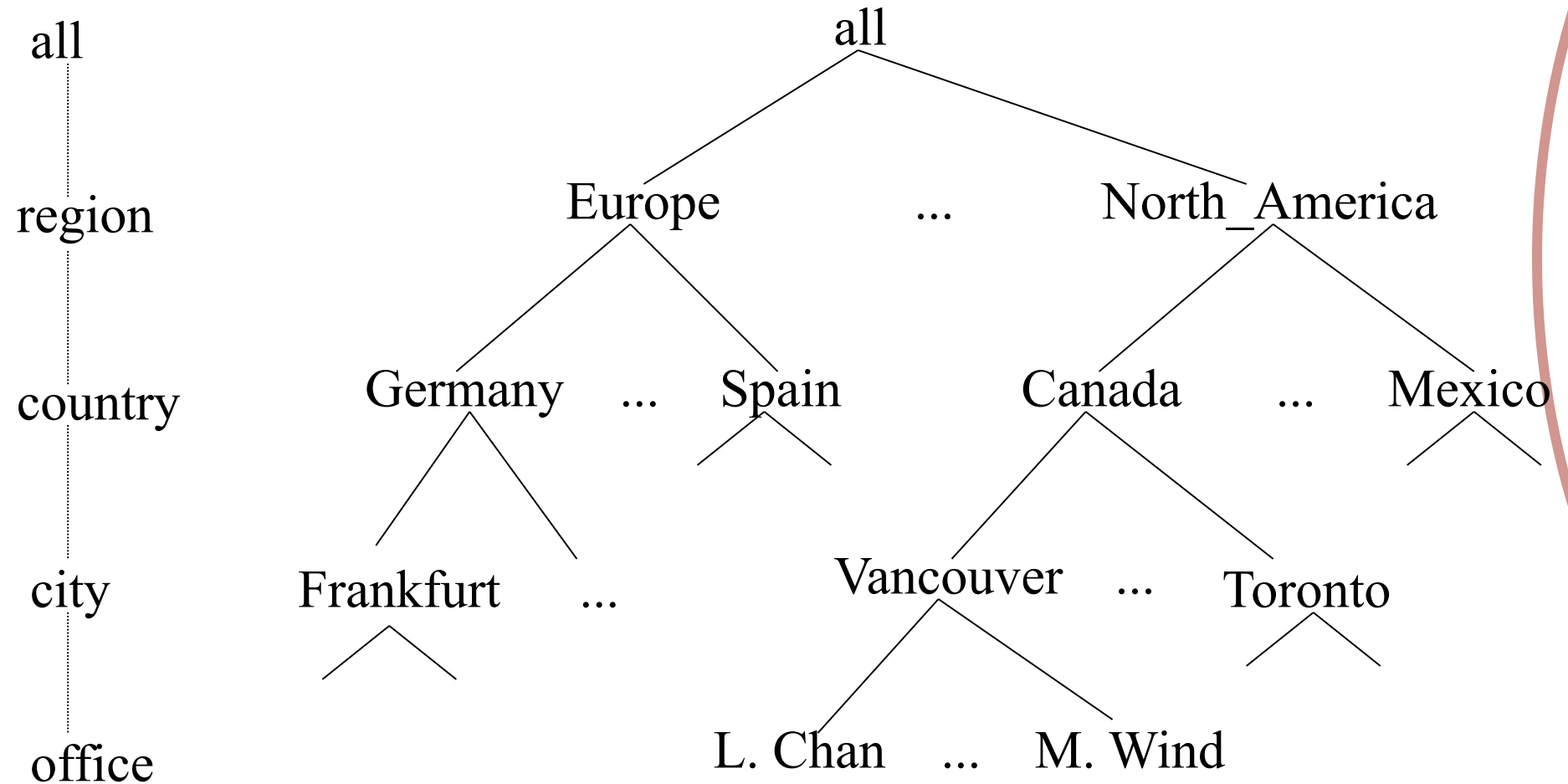
The Multi-Dimensional Model



Dimensional Modeling

- Dimensions organize attributes into stable analysis paths
- E.g., Time: day -> week -> month -> quarter -> year
- E.g., Product: SKU -> product line -> brand -> category
- Conformed dimensions reuse the same keys and labels across facts
- Role-playing dimensions reuse one table in different roles, e.g., order_date and ship_date

Concept Hierarchy: Location Dimension



Declaring Fact Grain

- Grain defines exactly what one row in the fact table represents
- Example: one row per sold order line, not one row per order
- The grain determines valid joins, uniqueness checks, and aggregations
- If the grain is ambiguous, double counting is likely
- Document grain and natural uniqueness before writing ETL

Grain: What It Is

- The grain is the business event level captured by the fact table
- It answers the question: what does one row mean?
- Grain is not the table name, source file, or dashboard metric
- Once declared, every key and measure must match that row meaning
- Changing grain later usually breaks joins, aggregates, and history

Common Grain Options

- Transaction grain: one row per event, e.g., order line, payment, click
- Periodic snapshot grain: one row per entity per period, e.g., daily inventory
- Accumulating snapshot grain: one row per process lifecycle, e.g., fulfillment case
- Aggregate grain: pre-summarized rows, e.g., product-day sales totals
- Factless grain: event existence only, e.g., attendance or promotion eligibility

Choosing the Right Grain

- Start with the lowest atomic grain needed for audit and flexible analysis
- Use aggregate grains only for performance marts, not as the only source of truth
- Avoid mixing grains in one fact table, such as order header and order line
- Check that the proposed grain has a stable unique key
- Write the grain statement before designing dimensions or ETL logic

Table Behavior Types

- Table behavior describes how rows are written and changed over time
- Append-only tables only insert new rows and preserve full event history
- Mutable tables allow updates and deletes when the latest state matters
- Late-arriving data reaches the warehouse after the business event happened
- These choices affect ETL, audits, reconciliation, and downstream trust

Append-Only vs. Mutable Tables

- Append-only: best for logs, transactions, CDC history, and replayability
- Mutable: best for current-state dimensions, operational mirrors, and corrections
- Append-only simplifies audit and backfill, but consumers may need latest-state views
- Mutable tables are easier for current-state queries, but they can hide history
- A common pattern is append-only raw layer plus curated mutable serving tables

Late-Arriving Data

- Late-arriving fact: event_time is earlier than load_time
- Late-arriving dimension: the fact arrives before the matching dimension row
- Do not drop these records; stage, track, and reconcile them
- Keep event_time separate from load_time
- Define unknown-key, backfill, and restatement policy

Measure Types and Aggregation Rules

- Additive: revenue, quantity - safe to sum across all dimensions
- Semi-additive: inventory_balance - sum across product, not across time
- Non-additive: conversion_rate - calculate from numerator and denominator
- Derived metrics should be certified in the semantic layer
- Store atomic components when possible; avoid averages of averages

Natural, Durable, and Surrogate Keys

- Natural key: source identifier such as customer_id
- Durable key: stable identity across systems and merges
- Surrogate key: warehouse-managed key used in fact joins
- Facts should join dimensions by surrogate key
- Late-arriving dimensions need an unknown-key policy

Slowly Changing Dimensions (SCD)

- Type 0: preserve original value, no changes after insert
- Type 1: overwrite old value when history is not required
- Type 2: keep historical versions with validity window and current flag
- Type 3: keep limited previous value in extra columns
- Choose SCD behavior by audit, finance, and analytical truth requirements

SCD Type 2 Example: Customer Region Change

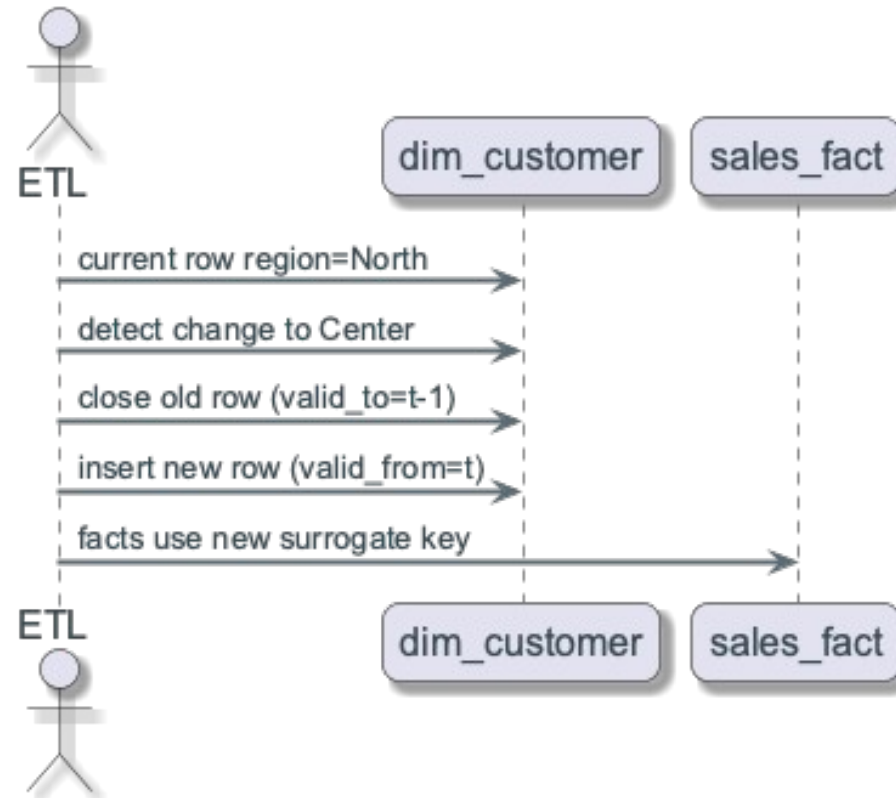
Customer moves from North to Center

Type 1: all historical sales are reclassified as Center

Type 2: past sales remain North, new sales use Center

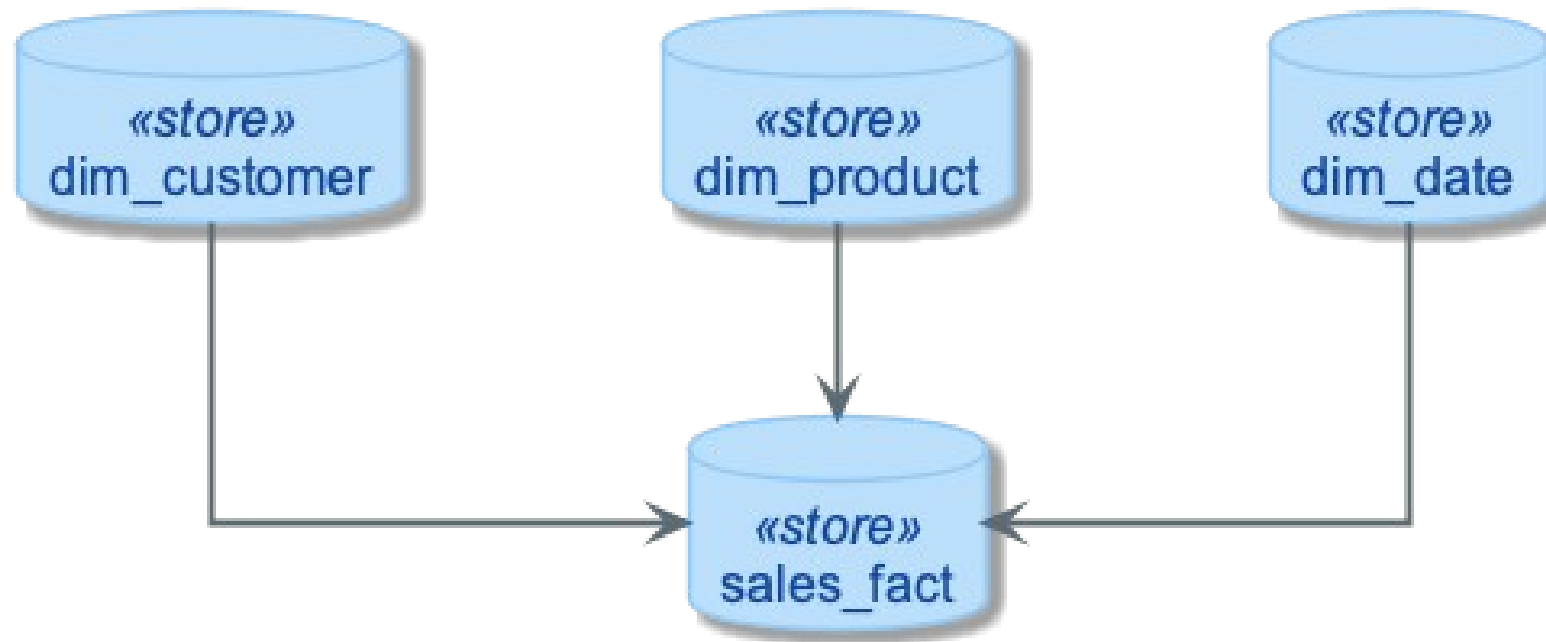
Fact rows join to the dimension version valid on the sale date

Finance and compliance usually require Type 2 for material attributes



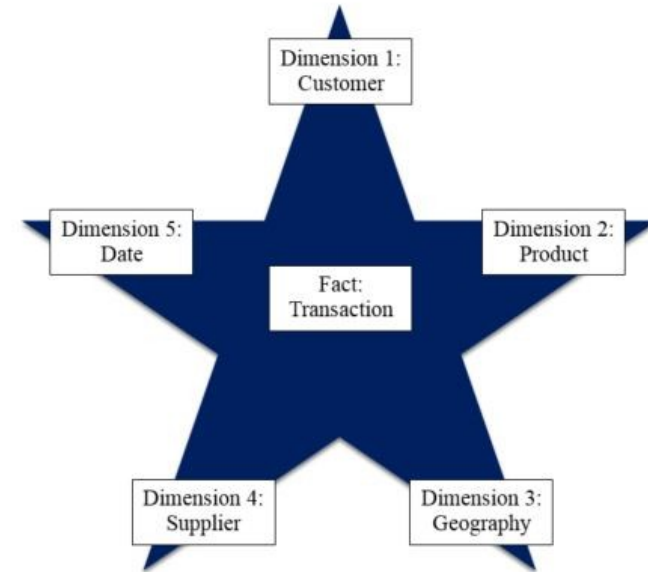
- SCD Type 2: versioned dimension rows

Star Schema: Default BI Design



The Classic Star Schema

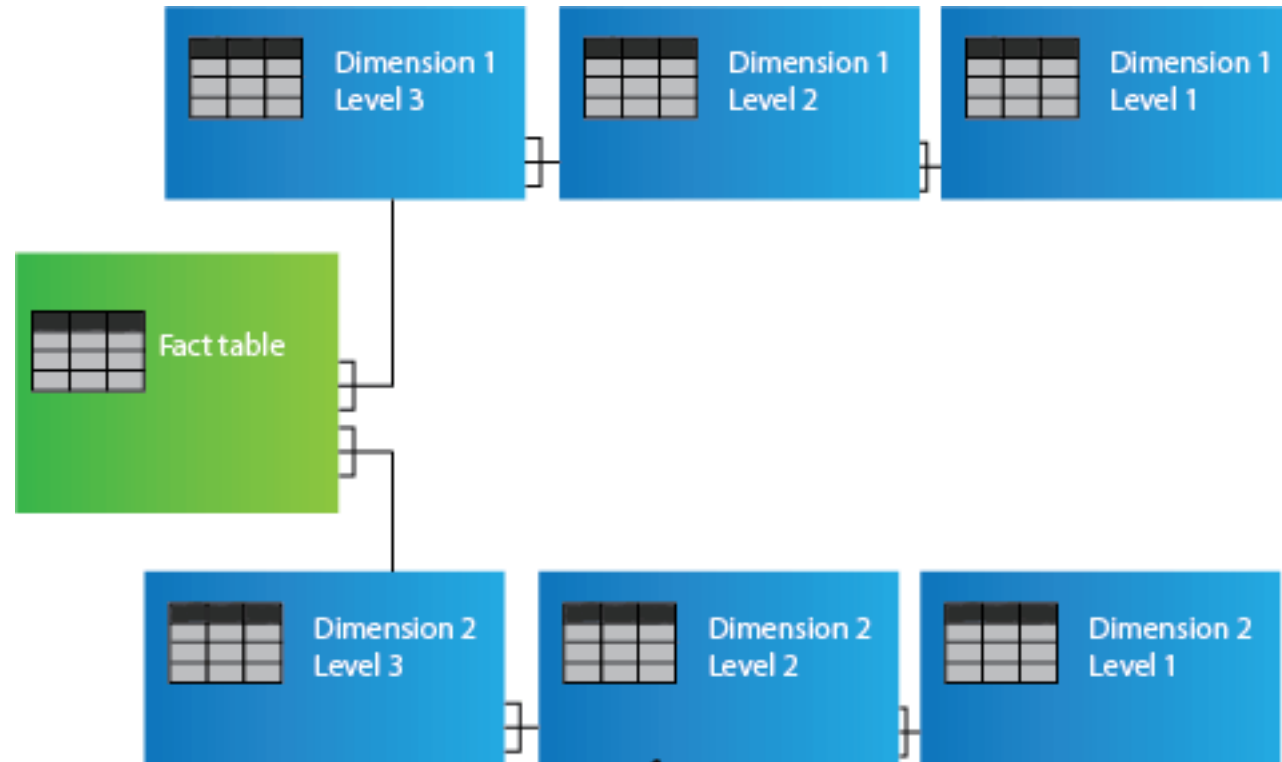
- A fact table sits at the center and connects to denormalized dimensions.
- Each dimension key appears once in the fact table primary/unique key.
- Dimension tables contain business labels, attributes, and hierarchies.
- Star schemas are usually easiest for BI tools, and query optimizers.
- Default to star unless normalization has a measurable benefit.



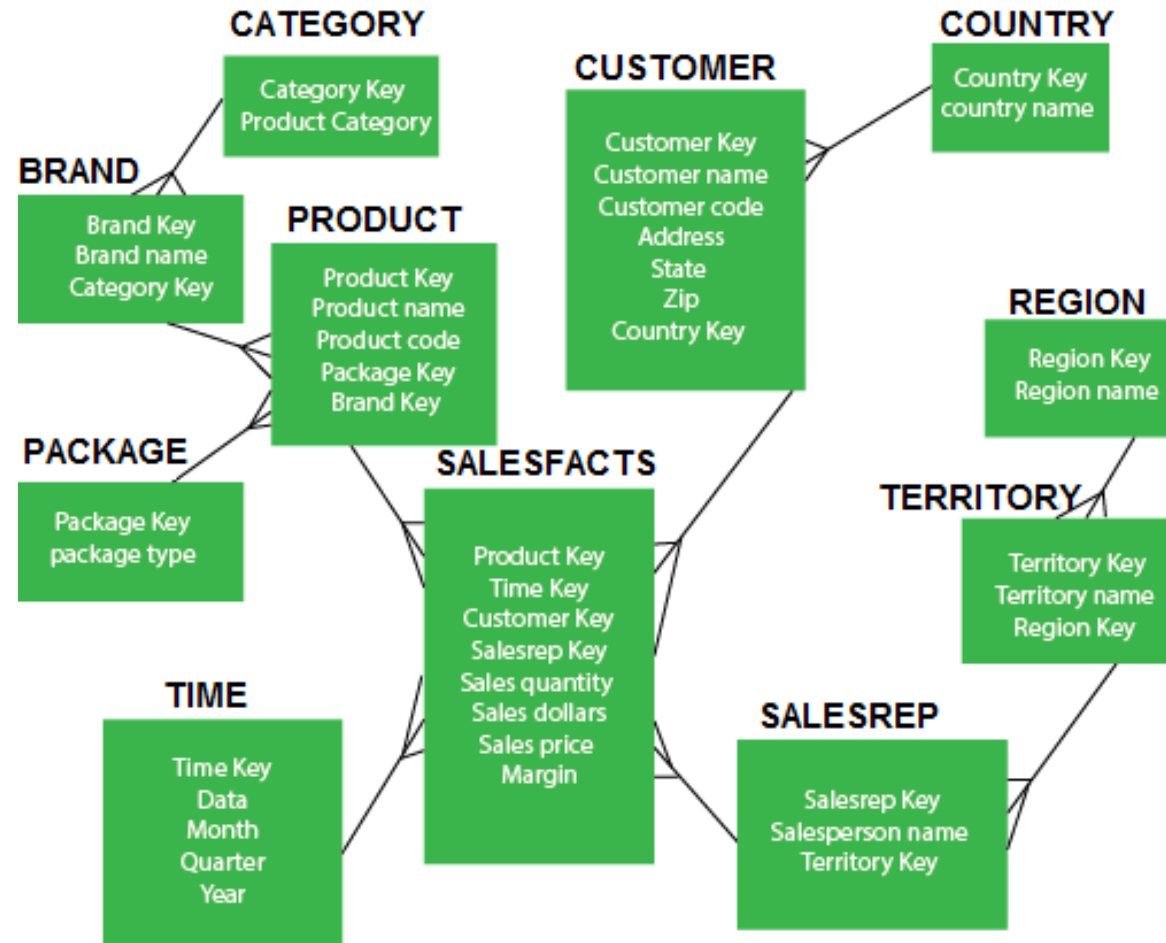
Snowflake Schema

- Snowflaking normalizes selected dimension attributes into sub-dimensions
- It can reduce redundancy in large, shared, hierarchical dimensions
- It adds joins, ETL complexity, and harder self-service BI
- Use selectively when maintenance or consistency benefits outweigh query cost
- Common compromise: normalized core plus star marts

Snowflake Schema: Normalized Dimensions

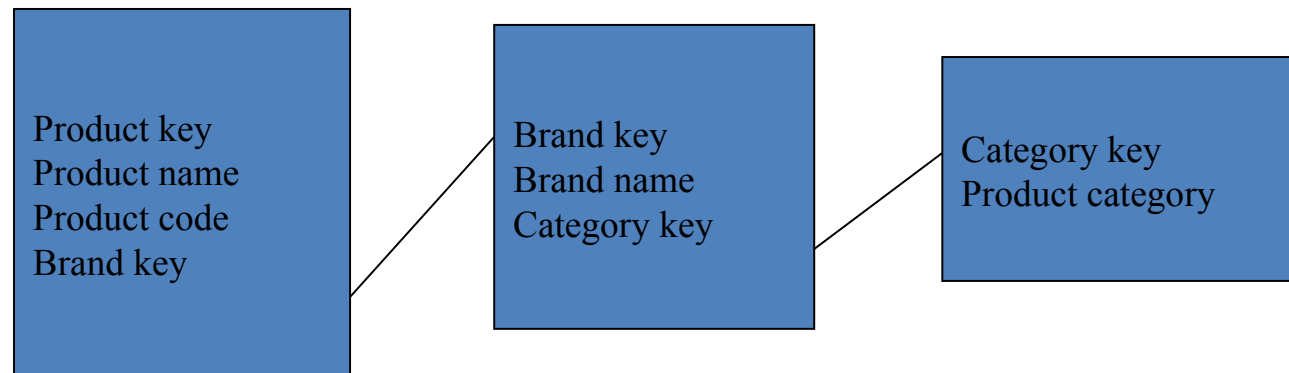


Snowflake Example: Product and Geography



Snowflaking a Product Dimension

- Split shared hierarchy attributes into reference tables
- Product stores brand_key and category_key
- Benefit: less repetition and stronger governance
- Cost: extra joins unless a mart denormalizes back



Incident: Full-Scan Dashboard Outage

- Dashboard refreshed every 5 minutes on a 1 TB atomic fact table
- A missing date predicate forced full scans on every refresh
- Concurrent scans saturated warehouse slots and delayed other pipelines
- Root cause: no model-level or semantic-layer guardrail for partition filters
- Fix: enforce required time predicates for large facts and monitor partitions scanned

Partitioning and Clustering Strategy

- Partition large facts by the dominant time filter, usually date_key
- Align partition granularity with query patterns and retention policy
- Cluster or sort on common filters such as customer_key or product_key
- Avoid tiny partitions and small files; they add metadata overhead
- Validate design with bytes scanned, partitions read, and p95 latency

Partition Pruning Cost Model

$$\text{ScanCost} = s \cdot |F|$$

$|F|$: total fact size

s : selected partition fraction

Example: $|F| = 1$ TB, one-day filter over 365 days ->

$s = 1/365$ - Scan drops from 1 TB to about 2.74 GB,

Cost and runtime often drop proportionally

Join Cost Intuition

$$\text{JoinWork} \approx O(|F_{pruned}| + \sum |D_i|)$$

Pruned fact size is usually dominant term

Broadcast joins help when dimensions fit memory

Large dimensions force shuffle and spill

Architecture Evolution: From Model to Contract

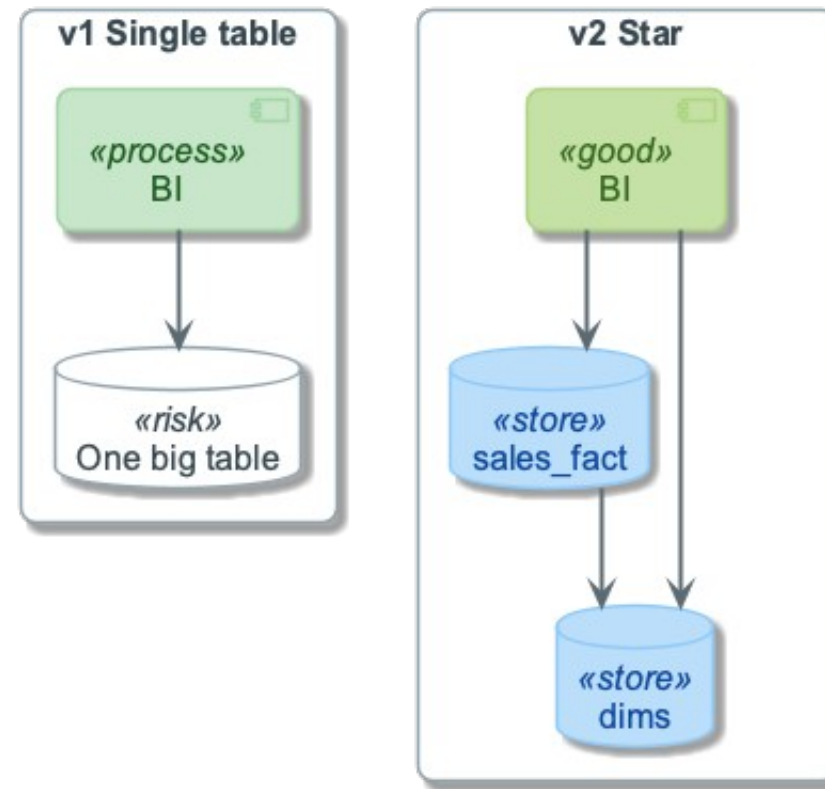
v1: one large table and manual metric definitions

v2: star schema, conformed dimensions, and semantic contracts
Add SCD2, bridges, and snowflaking only when needed

Migrate with compatibility views to avoid breaking dashboards

Version, test, document, and monitor schema changes

- Architecture evolution



Governance Controls for Large Facts

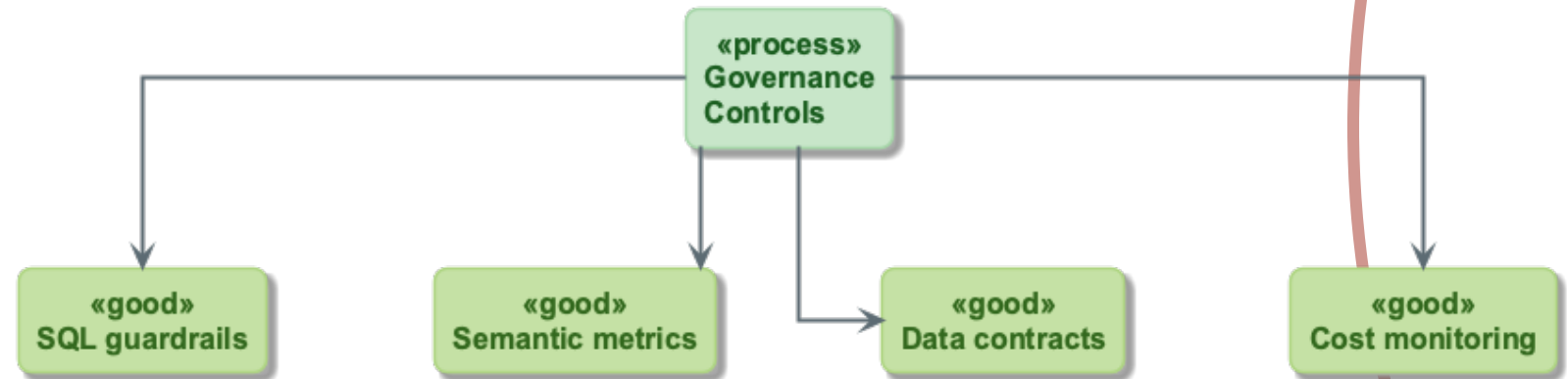
SQL guardrail: block large-fact queries without required time filters

Semantic layer: certified metric formulas, grain, filters, and owners

Data contracts: required keys, null policies, and uniqueness checks

Quality tests: freshness, duplicates, and reconciliation

Cost monitoring: bytes scanned, slots consumed, and query p95



Operational Metrics and Review Checklist

- Bytes scanned and partitions read per dashboard refresh
- Query latency p50/p95, queue time, and spill indicators
- Duplicate fact rows, unmatched dimension keys, and unknown-key rate
- Freshness lag, late-arriving volume, and reconciliation differences
- Compaction backlog, small-file count, and growth of the largest facts